1 Pushdown Automata

A pushdown automata (PDA) is:

- An NFA with a stack.
- Stack: an infinite LIFO memory.
- Equivalent to CFGs.

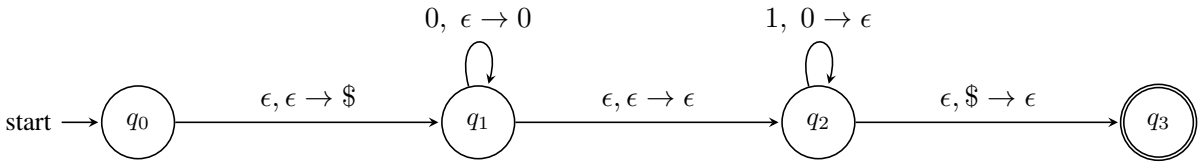
Definition A PDA is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$ where

- Q is the finite set of states.
- Σ is the input alphabet.
- Γ is the stack alphabet.
- $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \to 2^{Q \times \Gamma_\epsilon}$ is the transition function
- q_0 is the start state.
- $F \subseteq Q$ is the set of accepting states.

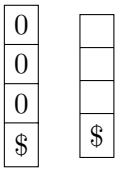
Stack notation $x \to y$: pop x, push y.

Example

- $L_1 = \{0^n 1^n \mid n \geq 0\}$



$w = 000111$:



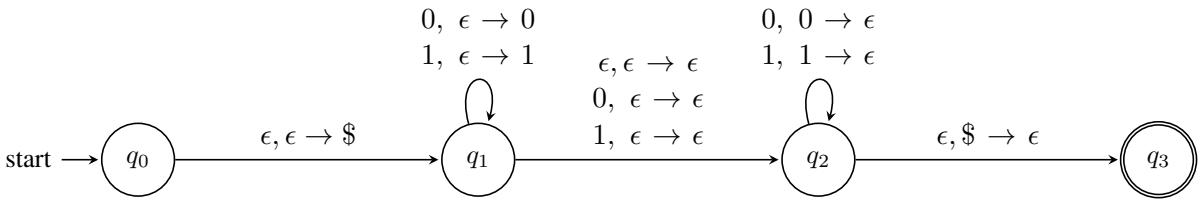
- $L_2 = \{0^m 1^n \mid m \leq n\}$

  Same PDA as above, just add a self-loop over q_3 with $1, \epsilon \to \epsilon$.

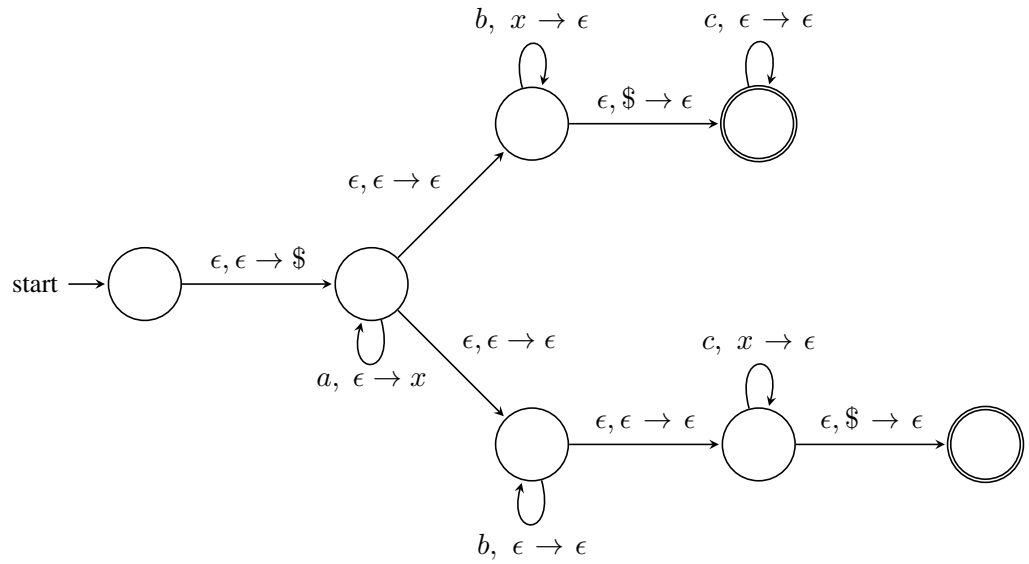
- $L_3 = \{0^m 1^n \mid m \geq n\}$

  Take the PDA for L_1, make q_2 an accepting state. You can remove q_3.

- $L_4 = \{w \mid w = w^R\}$ (eg. $1001, 10101, 11011, \ldots$)



- $L_5 = \{a^i b^j c^k \mid i = j \text{ or } i = k\}$
  $\Sigma = \{a, b, c\}, \Gamma = \{\$, x\}$



- $L_6 = \{w \mid n_0(w) = n_1(w)\}$. Exercise.

What you can't do with a 1-stack PDA

Example

- $L = \{a^n b^n c^n \mid n \geq 0\}$
- $L = \{vv \mid v \in \{0, 1\}^*\}$

Theorem 1.1 *PDAs and CFGs are equivalent.*

Corollary 1.2 *Every regular language is context-free, since every NFA is also PDA.*

Remark Deterministic PDA is not equivalent to PDA. The languages accepted by DPDA are a subset of CFLs that can be generated by non-ambiguous grammars.

2 Pumping Lemma for CFLs

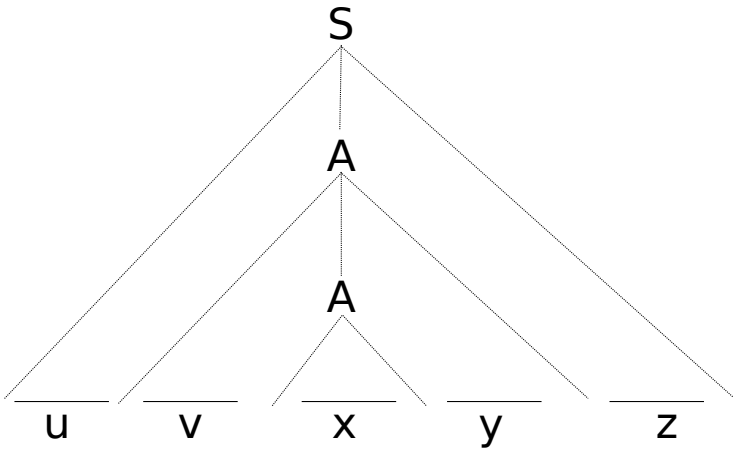
Consider a derivation in a CFG. A cycle in a derivation is something like:

$$S \stackrel{*}{\Longrightarrow} \ldots A \ldots \stackrel{+}{\Longrightarrow} \ldots A \ldots \stackrel{*}{\Longrightarrow} w$$

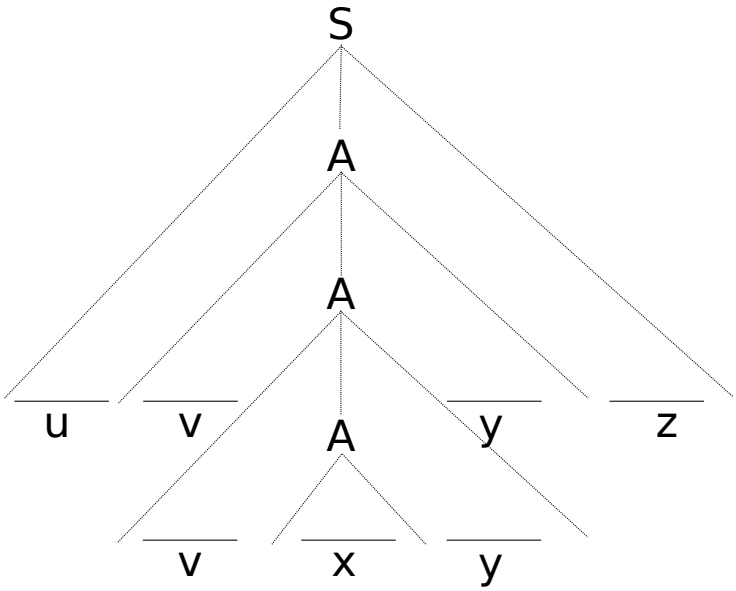
More accurately:

$$S \stackrel{*}{\Longrightarrow} uAz \stackrel{+}{\Longrightarrow} uvAyz \stackrel{*}{\Longrightarrow} uvxyz$$

Consider the derivation tree:



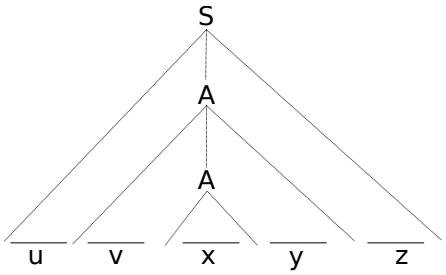
If we copy and paste the subtree under the first A in place of the second one, we get:



Or, in general:

$$uv^ixy^iz \in L$$

$\forall i \geq 0$. Note that $uxz \in L$ as well.



So, what is the length of the string that would guarantee such a cycle in its derivation?

First of all, if the height of the tree is $> n$, $n = |V|$, then some path in the tree must contain a repeated variable. Assume G is in CNF (i.e. the derivation tree is binary). Then, if $|w| \geq 2^{n-1} + 1$, its derivation tree will have some repeated variables on the same path.

2.1 Pumping lemma

Let L be a CFL. Then there is an integer p such that any $w \in L$, $|w| \geq p$, can be written as $w = uvxyz$, where $u, v, x, y, z \in \Sigma^*$.

- $uv^ixy^iz \in L$
- $|vy| > 0$
- $|vxy| \leq p$

Proof If L is CF, it can be generated by some grammar $G = (V, \Sigma, P, S)$ in CNF. Let $n = |V|$.

- The first condition follows from the previous argument.
- The second condition follows from the fact that G has no unit- or ϵ-productions.
- To prove the third part, we can assume that the repeated variable is in the bottom $n + 1$ variables of the tree, since a repetition has to occur on any path of $n + 1$ variables. Then, $|vxy|$ is at most 2^n. That is, for $p = 2^n$, $|vxy| \leq p$. ∎

Example Show that L is not CF:

- $L_1 = \{a^nb^nc^n \mid n \geq 0\}$

  Assume that L is CF and let p be its pumping length. Consider $w = a^pb^pc^p$.

  By the third condition of pumping lemma, the cycle part (v, y) cannot contain $a\&c$ at the same time ($|vxy| \leq p$, Hence whatever characters $v\&y$ may have, if we pump up, we will have an unequal number of a, b, c.

- $L_2 = \{a^mb^mc^n \mid n \geq m\}$

  Assume L is CF and let p be its pumping length. Consider $w = a^pb^pc^p$.

  There are two possibilities regarding v, y:

  - v, y consist of a & b, but no c.
  - v, y have some c, but no a. (since $|vxy| \leq p$)

  In the first case, pump up, and $uv^ixy^iz \notin L$.

  In the second case, pump down, and $uxz \notin L$.

- $L_3 = \{vv \mid v \in \{0, 1\}^*\}$

  Assume L is CF and let p be its pumping length. Consider $w = 0^p1^p0^p1^p$.

- $L_4 = \{0^{n^2} \mid n \geq 0\}$

  Assume L is CF and let p be its pumping length. Consider $w = 0^{p^2}$.

  We must have $vy = 0^k$ for some $k > 0$. Then, $uv^ixy^iz = 0^{p^2+k} \notin L$ (since $k \leq p$ and $p^2 + k \leq (p + 1)^2$.

- $L_5 = \{a^ib^jc^k \mid k = max(i, j)\}$

  Consider $w = a^pb^pc^p$. Again there are two possibilities:

  - v, y consist of a & b, but no c.
  - v, y have some c, but no a. (since $|vxy| \leq p$)

  In the first case, pump up, and $uv^ixy^iz \notin L$.

  In the second case, pump down, and $uxz \notin L$.

- $L_6 = \{0^i 1^j \mid i = j^2\}$
  Consider $w = 0^{p^2} 1^p$.
  - If v and y consist of the same symbol (eg $v = 0^k$ and $y = 0^l$), obviously $uv^i xy^i z \notin L$.
  - If either v or y is mixed (i.e. has both 0 and 1), then if we pump up, we will have mixed 0s and 1s, $uv^i xy^i z \notin L$.
  - The only plausible alternative is $v = 0^k,\ y = 1^l$ for some $k, l > 0$. If we pump up: $uv^i xy^i z = 0^{p^2+k} 1^{p+l}$. Note that $l \geq 1$. $p^2 + k < (p + l)^2$ for any $k < p$. Hence $uv^i xy^i z \notin L$.

3 Closure Properties of CFLs

Theorem 3.1 *CFLs are closed under union, concatenation, and star (Kleene's closure).*

Proof Let L_1 and L_2 be CFLs generated by $G_1 = (V_1, \Sigma_1, P_1, S_1)$ and $G_2 = (V_2, \Sigma_2, P_2, S_2)$.

- **Union:** $L_1 \cup L_2$ is generated by $G_3 = (V_3, \Sigma_3, P_3, S_3)$, where:
$$V_3 = V_1 \cup V_2 \cup \{S_3\}$$
$$\Sigma_3 = \Sigma_1 \cup \Sigma_2$$
$$P_3 = P_1 \cup P_2 \cup \{S_3 \to S_1 \mid S_2\}$$
- **Concatenation:** $L_1 \cdot L_2$ is generated by $G_3 = (V_3, \Sigma_3, P_3, S_3)$, where:
$$V_3 = V_1 \cup V_2 \cup \{S_3\}$$
$$\Sigma_3 = \Sigma_1 \cup \Sigma_2$$
$$P_3 = P_1 \cup P_2 \cup \{S_3 \to S_1 S_2\}$$
- **Star:** L_1^* is generated by $G_3 = (V_3, \Sigma, P_3, S_3)$, where:
$$V_3 = V_1 \cup \{S_3\}$$
$$P_3 = P_1 \cup \{S_3 \to S_1 S_3 \mid \epsilon\}$$

∎

Theorem 3.2 *CFLs are* **not** *closed under intersection.*

Proof by counter example. Consider:
$$L_1 = \{a^m b^m c^n \mid m, n \geq 0\}$$
$$L_2 = \{a^m b^n c^n \mid m, n \geq 0\}$$
which are context-free. Note that:
$$L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}$$
which is not context-free. ∎

Corollary 3.3 *CFLs are* **not** *closed under complementation.*

Proof Given that $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$, and that CFLs are closed under union but not under intersection, they cannot be closed under complementation. If they were closed under complementation, they would also be closed under intersection, which is not true. ∎

Theorem 3.4 *CFLs are closed under intersection with regular languages. If L_1 is CFL, and L_2 is RL, then $L_1 \cap L_2$ is also context-free.*

Proof Let L_1 be accepted by some PDA $M_1 = (Q_1, \Sigma, \Gamma, \delta_1, p_0, F_1)$ and L_2 be accepted by some NFA $M_2 = (Q_2, \Sigma, \delta_2, q_0, F_2)$. Take the "cartesian-product machine":
$$M = (Q_1 \times Q_2, \Sigma, \Gamma, \delta, (p_0, q_0), F_1 \times F_2)$$
where δ runs δ_1 and δ_2 in parallel and the stack is used by δ_1. ∎

4 Decision Algorithms for CFLs

- **whether $L(G)$ is empty:** Run the "useless symbol removal" algorithm. If the start variable is useless, then $L(G)$ is empty.
- **whether $L(G)$ is finite/infinite:** Convert the grammar into CNF. If the productions

  $$A \to BC$$

  give a cycle, then $L(G)$ is infinite. Else, it is finite.
- **whether $w \in L(G)$:** Convert the grammar into GNF.

  $$A \to a\alpha$$

  where $a \in \Sigma$, $\alpha \in V^*$. Check exhaustively whether w can be produced from S.

There are no decision algorithms to answer:

- If two CFGs are equivalent.
- If the complement of a CFL is also CF.
- If a given CFG is ambiguous.